Lab 07 — Networking: making containers talk to each other

Theory — how three containers that do not know each other's addresses find each other by name, what "publishing a port" really means, and why `localhost` is pitfall number one.

Objectives

- Understand what network a container gets, and what rootless mode changes about it (`pasta`).
- Tell the default network from a **user-defined** network, and know why the latter has DNS.
- Read and write `-p`, `-P`, `EXPOSE`, `--network`, `--network-alias`.
- Diagnose the three classic mistakes: `localhost`, unpublished port, unresolved name.
- Build the **web → api → db** chain by name, and understand what a *pod* is.

---

1. The problem: three processes, three islands

In lab 06, `db` ran alone. As soon as `api` and `web` join, nginx must find the API, and the API must find PostgreSQL. But every container is isolated: it has its own network stack, its own address, and **its own `localhost`**. Inside the `api` container, `localhost:5432` means the API itself, not the database. It is the most common mistake every beginner makes, and plenty of veterans too.

> **→ LINUX**
> A **network namespace** gives a process its own network stack: its interfaces, its routing table, its firewall rules, its ports. Two namespaces can each have port 8080 open with no conflict. `lo` (127.0.0.1) exists in each one and leads only to itself. A container is, among other things, a network namespace — which is why `localhost` is a dead end there.

IP addresses do not help either: assigned at creation, they change when a container is recreated. You never hard-code a container IP. You need **names**.

2. What a container gets

On a classic Docker setup, the root daemon creates a **bridge** (`docker0`), plugs each container into it through a pair of virtual interfaces, and performs NAT towards the outside. The container gets a private IP (`172.17.0.x`) and reaches the Internet through the host's address.

> **→ NETWORK**
> A **bridge** is a software switch: interfaces plugged into it form a local network. A **veth** pair is a virtual cable with one end inside the container (`eth0`) and the other on the bridge. **NAT** (*network address translation*) rewrites the source address of outgoing packets to the host's, so that replies find their way back. Three kernel building blocks, assembled by the engine.

> **PODMAN**
> An ordinary user is not allowed to create a bridge or set up NAT. Under rootless, Podman therefore hands the default network to **`pasta`**, a user-space program that *copies* the host's address into the container: `ip addr` inside the container shows the same IP as your WSL, and `inspect` shows an empty `IPAddress`. **Bridge** networks remain possible (`--network podman`, or a network you create): they live in a private namespace of Podman's, and `rootlessport` relays the published ports — hiding the real client's address along the way.

Network	How to get it	Container address	DNS between containers
default (pasta)	specify nothing	the host's, copied	no
`podman` (bridge)	`--network podman`	`10.88.0.x`	**no** (`dns_enabled: false`)
user-defined (bridge)	`podman network create stack` then `--network stack`	`10.89.x.x`	**yes** (`aardvark-dns`)
host	`--network host`	the host's, with no isolation	—
none	`--network none`	none, only `lo`	—

3. The user-defined network and its DNS

This is the golden rule of this lab: **one application = one network created for it**. `podman network create stack` creates a private bridge with a subnet (`10.89.0.0/24`) and, above all, an **internal DNS server** (`aardvark-dns`, on the gateway address `10.89.0.1`). Every container started with `--network stack` can reach the others **by container name**: `api` resolves to the IP of `api`, `db` to that of `db`. The container's `/etc/resolv.conf` points to `10.89.0.1`, with `search dns.podman`.

Docker works the same way: its default bridge has no DNS, its user-defined networks do. The default network is for a quick test; as soon as two containers must talk, you create a network.

A container can belong to **several** networks (`podman network connect admin api`) and carry **aliases** (`--network-alias backend`): the API is then reachable under two names. A network also isolates: a container on the `admin` network does not resolve `api` until `api` is connected to it.

> **PITFALL**
> DNS resolves a name to an IP **at connection time**. nginx, however, resolves the names in its `proxy_pass` directives **once, at startup**, and keeps the IP. Recreate `api` and give it a different address, and nginx keeps sending to the old one: `502 Bad Gateway`, `Host is unreachable` in its logs. Remedy: restart `web`, or configure nginx with a `resolver` directive and a variable. Spring Boot has no such problem: its connection pool opens new connections, hence resolves again.

4. Publishing a port

On a bridge network, a container is reachable only by its neighbours. For an **outside** client (your browser, `curl` from WSL) to reach it, you must **publish** a port: `-p 18080:80` asks the engine to listen on port 18080 of the host and relay to port 80 of the container.

Syntax	Effect
`-p 18080:80`	host `0.0.0.0:18080` → container `80`
`-p 127.0.0.1:18080:80`	only from the machine itself
`-P`	publishes every `EXPOSE`d port on random host ports (`39843`…)
`EXPOSE 8080` (Dockerfile)	**publishes nothing**: documentation, read by `-P` and by `podman ps`

`EXPOSE` is the classic misunderstanding: the image says "I listen on 8080", nothing more. Without `-p`, `curl localhost:8080` from the host fails, while containers on the same network reach port 8080 with no publishing at all. **Between containers, you do not publish**; you publish for the outside world.

> → SECURITY
> On Linux, ports **below 1024** are reserved for root. Under rootless, `-p 80:80` fails: `rootlessport cannot expose privileged port 80 … bind: permission denied` (or `pasta failed` on the default network), and the container stays in the `Created` state. Three ways out: publish on a high port (`-p 8080:80`), lower the limit (`sysctl net.ipv4.ip_unprivileged_port_start=80` , an administrator's decision), or put a root-run reverse proxy in front. In the workplace, a container **never** listens on 80 or 443 directly: that is the job of the load balancer or the Ingress.

> → WINDOWS / WSL
> A port published inside WSL is reachable from Windows at `http://localhost:18080` : WSL 2 automatically forwards the ports the distribution listens on to `localhost` on the Windows side — even with `-p 127.0.0.1:…` . Other machines on the network, however, do not see that port, unless you enable *mirrored* networking mode (`.wslconfig` , `networkingMode=mirrored`) or add a `netsh portproxy` rule.

5. host, none, and reaching the host

`--network host` removes network isolation: the container uses the host's stack directly, with no `-p` (it no longer means anything). Useful for a network diagnostic tool; dangerous for a service. And under rootless, an nginx run with `--network host` fails to open port 80: `bind() to 0.0.0.0:80 failed (13: Permission denied)` .

`--network none` provides only `lo` : no network access at all, inbound or outbound. It is the right setting for a batch job that must talk to nobody, or to check that an application has no hidden network dependency.

Finally, a container sometimes needs to reach a service **on the host** (a database installed in WSL, another published container). The name `host.containers.internal` designates the host from the default network — the equivalent of Docker Desktop's `host.docker.internal` .

6. Pods

> PODMAN
> A **pod** is a group of containers that share **the same network namespace**: the same IP, the same ports, the same `localhost` . Inside a pod, nginx reaches the API at `localhost:8080` , and `-p` is declared **on the pod**, not on the containers. It is exactly the Kubernetes model, whose vocabulary Podman reuses: `podman pod create --name stackpod -p 18090:80` , then `podman run --pod stackpod …` . An invisible *infra* container keeps the namespace open. Docker has no pods; `--network container:other` is the closest approximation.

Keep the nuance in mind: on a **network**, containers talk to each other by name; in a **pod**, through `localhost` . Both models coexist, and the Kubernetes part of this series builds on the second.

7. In the workplace

- **One network per application**, with three services on it: `web` , `api` , `db` . Only `web` publishes a port. `db` is **never** published in production; during development, `-p 127.0.0.1:5432:5432`

lets you plug in a SQL client.

- **The API finds the database by name**: `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app`. A URL with `localhost` is the sign of a workstation configuration copied into a container unchanged.
- **nginx acts as reverse proxy**: `/` serves the front end, `/api/` is relayed to `api:8080`. The browser sees a single origin, which avoids CORS, and the API is not exposed directly.

> **→ HTTP**
> **CORS** (*cross-origin resource sharing*) is a browser rule: a page served by `localhost:18080` cannot call `localhost:8080` unless the API consents through headers. By placing the front end and the API behind the same nginx, on the same origin, the problem disappears — one of the reasons reverse proxies exist, along with TLS and caching.

> **→ SPRING BOOT**
> The connection pool (HikariCP) opens a few connections at startup and creates more over time. If `db` is not ready yet, Spring Boot fails at startup: `Connection refused`. Hence the importance of start-up order and *healthchecks* (lab 09), and of the `spring.datasource.hikari.initializationFailTimeout` option to tolerate a database that arrives late.

Remember

- `localhost` inside a container is the container: you address the others by **name**.
- The default network (pasta under rootless) and the `podman` network have no DNS; a **user-defined** network does: one application = one network.
- `-p host:container` publishes for the outside world; between containers on the same network, nothing to publish. `EXPOSE` publishes nothing.
- Under rootless, no port < 1024: `-p 8080:80`, or a reverse proxy in front.
- nginx freezes the IPs of its `proxy_pass` at startup: recreated container = `502`, restart nginx.
- `--network host` removes isolation, `--network none` removes the network, `host.containers.internal` designates the host.
- A **pod** shares the network namespace: `localhost` between containers, `-p` on the pod — the Kubernetes model.

Vocabulary

network namespace: a container's private network stack. — **bridge**: virtual switch connecting containers. — **veth**: virtual cable container ↔ bridge. — **NAT**: address rewriting to go out through the host. — **pasta**: Podman's default rootless network, copies the host's address. — **rootlessport**: relay for published ports under rootless. — **aardvark-dns**: internal DNS of user-defined networks. — **publish (`-p`)**: relay a host port to the container. — **`EXPOSE`**: documentary declaration of a port. — **alias**: additional name of a container on a network. — **pod**: group of containers sharing the network namespace. — **reverse proxy**: server that relays requests to internal services.